



In-memory compression of Awkward Arrays with Coffea

Peter Fackeldey¹, Iason Krommydas²,
Princeton University¹, Rice University²



Memory Limits Kill Your Jobs

Chunk too big → crash. Chunk too small → slow.

Limited memory forces tiny chunks, causing:

- Task scheduling overhead
- Inefficient use of vectorized kernels
- Python overhead

Reduced memory footprint → more robust & faster analysis

How Awkward Arrays Are Stored in Memory

Nested data (e.g. per-event jet lists) are stored as flat 1D buffers:

- **Offsets:** where each event starts/ends
- **Values:** all data flattened into one array

100k events → just 2 contiguous buffers (not 100k Python lists!).

Each buffer has a **unique key**. This maps naturally to a key-value store for compression and lazy lookup.

User View

Jet-momenta for variable number of Jets

Event 0: [45.2, 32.1, 12.5]
 Event 1: [67.8]
 Event 2: [23.0, 91.4]

1D Buffers

In-memory Layout of Awkward Arrays

Offsets: [0, 3, 4, 6]
 Values: [45.2, 32.1, 12.5, 67.8, 23.0, 91.4]

Key-Value Store

NEW!

Key-Value Cache Store for 1D Buffers

offsets/jets/pt → Offsets
 data/jets/pt → Values

Benefits of a Key-Value Store for Buffers:

- Full control over caching strategy
- Only accessed buffers are loaded into the key-value store (VirtualArrays)
- Transparent to users

Solution: BufferCache for Awkward Arrays

New in coffea: plug any key-value store (BufferCache) underneath your arrays.

```

1 from coffea.nanoevents import NanoAODSchema, NanoEventsFactory
2 from coffea.nanoevents.mapping import BufferCache
3
4 buffer_cache = BufferCache(...)
5
6 factory = NanoEventsFactory.from_root(
7     {path: "Events"},
8     mode="virtual",
9     buffer_cache=buffer_cache,
10 )
  
```

Three strategies below:

1. In-Memory Compression

Compress buffers before caching. Decompress only on access → larger chunks, same memory limit.

Example with Blosc:

```

1 from coffea.nanoevents.mapping import BufferCache
2 from numcodecs import Blosc
3
4 buffer_cache = BufferCache(
5     cache={},
6     codec=Blosc("zstd", clevel=1, shuffle=Blosc.BITSHUFFLE)
7 )
  
```

2. On-Disk Caching

Spill buffers to disk. Bypasses memory limits entirely — at the cost of I/O latency.

```

1 from coffea.nanoevents.mapping import BufferCache
2 from numcodecs import Blosc; import zict
3
4 buffer_cache = BufferCache(
5     cache=zict.File("cache_dir"),
6     codec=Blosc("zstd", clevel=1, shuffle=Blosc.BITSHUFFLE)
7 )
  
```

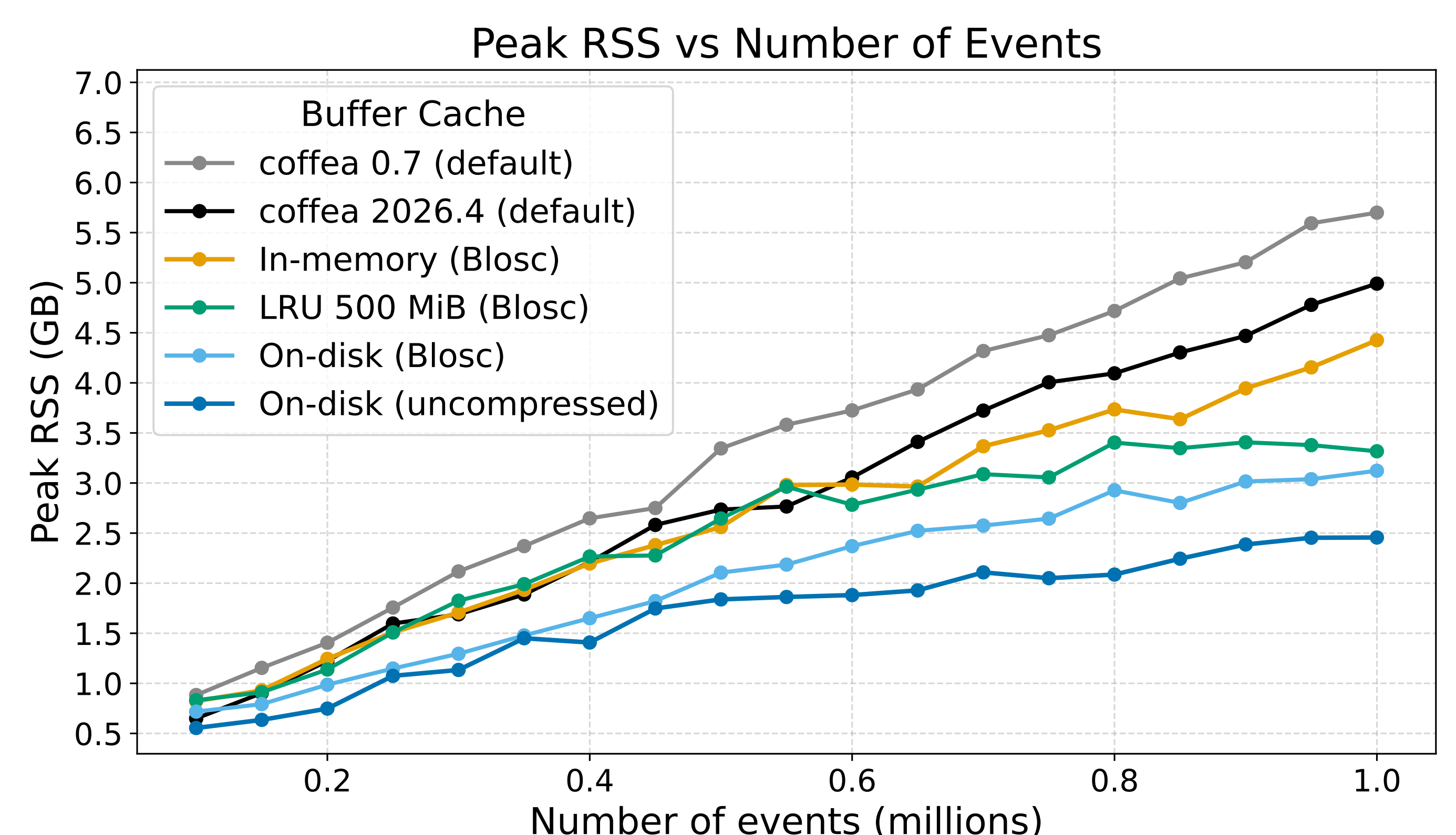
3. Custom / Tiered Caches

Any MutableMapping[str, bytes] works — e.g. tiered caches that spill to disk once a memory threshold is reached, using `zict.LRU` or similar.

Benchmarks

Comparing memory use for a typical coffea analysis, using the *Analysis Grand Challenge* with a CMS open data tt NanoAOD sample:

- No cache (default): coffea 0.7 and coffea 2026.4
- In-memory (Blosc)
- On-disk (zict.File, SSD ≈10 GB/s read speed)
- Tiered LRU (zict.LRU)



Results

- Peak RSS **reduced by up to ≈2×** with on-disk caching
- In-memory compression also reduces peak RSS vs default, especially at higher chunk sizes
- Runtime overhead ≈1..30% (depends on cache strategy and chunk size)
 - Mitigated by larger chunks + codec tuning
 - On-disk performance is *highly sensitive to disk bandwidth* → **understand your setup!**

→ **Try coffea≥2026.4 if memory limits slow you down!**